Studio Nx

백엔드 178 — Tag 모델 컬럼

2026-10-15

Backend(FastAPI/Python) 178 — `Tag` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/tag.py` (`Tag` 클래스)
작업	S1-07 샘플 라이브러리: ORM 모델 컬럼 (`docs/DB.md` §3.2, `studionx/db/02_phase2_sequencer.sql`)
작성	Claude (AI), 2026-10-15, 사용자 검토
이전 · 다음	177 `Sample` 모델 · 179 `SampleTag` 모델

1. 하는 일

MySQL의 `tags` 테이블(샘플에 붙이는 태그)을 파이썬 클래스 `Tag`로 옮겨 적습니다. 태그는 분류(`category`)별로 이름이 정해지고, `is_system`은 기본으로 제공하는 태그인지 나타냅니다. 샘플과 태그는 `sample_tags` 표로 이어집니다(다대다).

tag_id	category	name	is_system
1	`GENRE`	`Lo-fi`	1
2	`INSTRUMENT`	`Kick`	1
7	`MOOD`	`Deep`	0

`category`	뜻
`GENRE`	장르입니다 (예: `Techno`, `Lo-fi`)
`INSTRUMENT`	악기입니다 (예: `Kick`, `Piano`)
`MOOD`	분위기입니다 (예: `Deep`, `Bright`)
`ETC` (DB 기본값)	그 밖입니다

2. 언제 쓰나

쓰는 곳	하는 일
S1-07 라이브러리 태그 필터 (FR-224)	`category`별로 태그 목록을 보여 주고, 고른 태그가 붙은 샘플만 거릅니다
S1-07 태그 추가 (FR-225)	새 태그를 만들거나(`is_system=0`) 기존 태그를 붙입니다. 같은 분류에서 같은 이름은 만들 수 없습니다
(기본 제공 태그)	`is_system=1`인 태그는 클릭으로 고를 수 있게 기본으로 넣어 둡니다

3. 코드

import (파일 맨 위, `SampleTag`와 함께 씁니다)

```
from datetime import datetime
from enum import StrEnum
from typing import TYPE_CHECKING
```

```
from sqlalchemy import BigInteger, Boolean, Enum, ForeignKey, Integer, String, text
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.session import Base
```

import	가져오는 것	쓰는 곳
`StrEnum`	값이 문자열인 열거형	`TagCategory`
`Integer`, `String`	SQL의 `INT`, `VARCHAR(n)`	`tag_id`, `name`
`Boolean`	SQL의 `TINYINT(1)`	`is_system`
`Enum`	SQL의 `ENUM(…)`	`category`
`text`	SQL 문장을 그대로 넘깁니다	기본값 `1`, `CURRENT_TIMESTAMP(3)`
`DATETIME` (MySQL 전용)	`DATETIME(3)`	`created_at`
`relationship`	FK를 파이썬 속성으로 오가게 함	`tag.samples`

클래스 (칸 선언)

```
class TagCategory(StrEnum):
    GENRE = "GENRE"
    INSTRUMENT = "INSTRUMENT"
    MOOD = "MOOD"
    ETC = "ETC"


class Tag(Base):
    __tablename__ = "tags"

    tag_id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(50))
    category: Mapped[TagCategory] = mapped_column(Enum(TagCategory), server_default=TagCategory.ETC)
    is_system: Mapped[bool] = mapped_column(Boolean, server_default=text("1"))
    created_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP
(3)"))

    samples: Mapped[list["Sample"]] = relationship(secondary="sample_tags", back_populates="tags")
```

SQL과 한 줄씩 비교

SQL (`02_phase2_sequencer.sql`)	파이썬
`tag_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY`	`Integer`, `primary_key=True`, `autoincrement=True`
`name VARCHAR(50) NOT NULL` + `UNIQUE KEY uq_tags_category_name (category, name)`	`Mapped[str]`, `String(50)`, `UniqueConstraint`는 표에 있음(칸이 아님)
`category ENUM('GENRE','INSTRUMENT','MOOD','ETC') NOT NULL DEFAULT 'ETC'`	`Enum(TagCategory)`, `server_default=TagCategory.ETC`
`is_system TINYINT(1) NOT NULL DEFAULT 1`	`Mapped[bool]`, `Boolean`, `server_default=text("1")`
`created_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)`	`server_default=text("CURRENT_TIMESTAMP(3)")`

문법 풀이

식	뜻
`Boolean` + `server_default=text("1")`	`TINYINT(1)`을 파이썬 `bool`로 다룹니다. DB가 `1`을 채우면 `True`로 읽힙니다
`Enum(TagCategory)`	정해진 값(`GENRE` 등)만 쓰게 합니다. `TagCategory.GENRE == "GENRE"`가 참입니다
`UNIQUE(category, name)`	두 칸을 **합쳐서** 유일합니다. 같은 분류에서 같은 이름은 한 번만, 다른 분류라면 같은 이름도 됩니다 (`MOOD`의 `Deep`과 `GENRE`의 `Deep`)
`relationship(secondary=…)`	`sample_tags` 표를 거치는 다대다입니다. 반대쪽은 `Sample.tags`입니다

4. 흐름 (태그를 붙일 때, FR-225)

1. 태그를 찾습니다. 없으면 `Tag(name="Deep", category=TagCategory.MOOD, is_system=0)`을 만들어 `db.flush`합니다.
2. `SampleTag(sample_id=…, tag_id=…)`로 이어 줍니다(179번).
3. 같은 분류에서 같은 이름을 또 만들면 DB가 막습니다(`UNIQUE(category, name)`).

5. 왜 이렇게

선택	이유
`is_system`을 파이썬 `bool`로	화면에서 "기본 태그인지"만 보면 되므로 `1/0` 대신 `True/False`가 읽기 쉽습니다
`category` 기본값을 `ETC`로	DDL의 기본값과 맞췄습니다. 태그는 보통 분류를 적어 만들지만, 적지 않으면 `ETC`가 됩니다
`tag_id`는 부호 없는 `INT`	`BIGINT`가 아니라 `INT`입니다(DDL 그대로). 태그는 샘플보다 적게 쌓이므로 `Integer`로 충분합니다
`created_at`만 있음	태그는 이름과 분류를 거의 고치지 않으므로 `updated_at`이 없습니다 (DDL 그대로)

6. 시험 (`tests/test_tag_model.py` 중 `Tag` 부분, 4개)

실제 MySQL의 `tags` 테이블과 비교했습니다. 넣은 행은 시험이 끝나면 되돌립니다.

시험	확인한 것
칸이 DB와 같은지	칸 이름 5개, 순서, `NULL` 허용 여부가 실제 테이블과 같습니다
기본값	분류를 적지 않으면 `ETC`, `is_system`은 `True`, `created_at`은 채워집니다
같은 분류·같은 이름	두 번 만들면 DB가 거절합니다(`IntegrityError`)
다른 분류·같은 이름	`MOOD`의 `Deep`과 `GENRE`의 `Deep`은 둘 다 들어갑니다(행 2개)

전체 시험 871개가 통과했고, `ruff` 검사도 통과했습니다.